Integrated Complaint Monitoring and Status Dashboard

Design document · Smart India Hackathon 2026 · Draft v1 · 25 October 2026

1. Overview

Problem. AICTE receives complaints about technical institutions from students, faculty and others. Different Bureaus/Cells (the Regulation Bureau, RB; the Public Grievance Redressal Cell, PGRC; and others) handle them separately. Nobody has one live view of all cases, so two units can work on the same institution without knowing it. This causes duplicate effort, conflicting actions and delays.

Solution. A single-window web dashboard where authorized AICTE officials can see every institution-related complaint in one place, know which Bureau/Cell is handling it, follow its full history, and get an automatic warning when more than one unit is dealing with the same institution.

Key idea (innovation). An overlap-detection engine that combines exact rules (same Institution ID) with fuzzy matching (similar institution names) and optional text similarity (similar complaint wording). It shows the message “Potential Overlapping Cases – Coordination Required” even when case references or descriptions differ.

Expected outcome. One source of truth for complaints, less duplicated work, better coordination, timely disposal, and more accountability and transparency.

2. Scope

In scope for the 3-day prototype (MVP)

- Login with roles (Admin, Bureau Officer, Senior Management, Regional Officer)
- Institution profile with consolidated complaint history
- Case creation, case detail page, and full action history
- Transfer and escalation of cases between Bureaus/Cells
- Overlap detection with alerts and case linking
- Real-time dashboard with KPIs and charts
- Search and filters
- Alerts for overlaps, pending cases and deadlines (SLA)
- Reports downloadable as CSV

- Realistic fake data for the demo

Future scope (mention in slides, build only if time remains)

- PDF reports, email/SMS notifications, PostgreSQL, single sign-on with AICTE systems, integration with the live AICTE grievance portal, machine-learning-based matching.

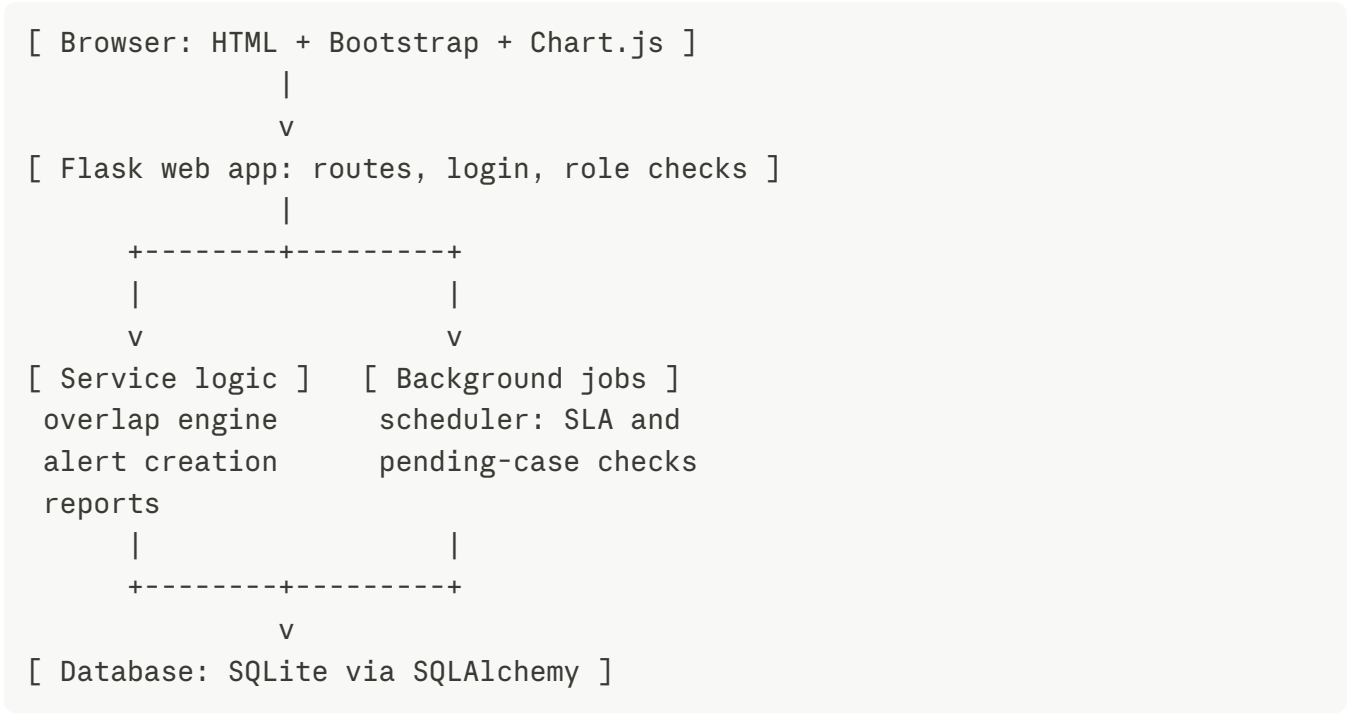
3. Users and Roles

Role	Who	What they can do
Admin	System administrator	Manage users, bureaus, categories and SLA rules; see everything
Bureau Officer	Staff of RB, PGRC or another Bureau/Cell	Create and update cases of their own unit; transfer or escalate; see overlap alerts affecting them; view (read-only) other units' cases for the same institution
Regional Officer	Regional Office/Centre staff	See cases of institutions in their region; add remarks
Senior Management / Competent Authority	Senior officials	Read-only access to all dashboards, reports and escalations

Rule: only logged-in, authorized users can see anything. Every page checks the user's role before showing data.

4. System Architecture

The system has four simple layers. Each layer only talks to the one next to it.



- **Browser layer** shows pages and charts.
- **Flask layer** receives requests, checks who the user is, and decides what to show.
- **Service layer** holds the smart parts: overlap detection, alert rules and report building. Keeping this separate from the web pages makes it easier to test and explain.
- **Database layer** stores everything. SQLite is used for the prototype because it needs no setup; the design works unchanged on PostgreSQL later.

5. Technology Stack and Why

Need	Choice	Reason
Backend	Python + Flask	Simple, fast to build, easy for AI coding tools to generate
Database	SQLite (prototype), PostgreSQL (production)	Zero setup now, scalable later
Database access	SQLAlchemy	Write Python instead of raw database commands
Pages	Jinja2 templates + HTML + Bootstrap	Ready-made, good-looking, responsive layout
Charts	Chart.js	Easy dashboard charts
Login and roles	Flask-Login	Standard, well documented

Need	Choice	Reason
Form safety	Flask-WTF	Protects forms against CSRF attacks
Similar-name matching	RapidFuzz	Fast fuzzy text comparison
Text similarity (optional)	scikit-learn TF-IDF	Light AI-assisted layer, no heavy model
Scheduled checks	APScheduler	Runs alert checks automatically
Reports	pandas	One-line CSV/Excel export
Fake data	Faker	Realistic demo data
Teamwork	Git + GitHub	Shared code and safe rollback

6. Data Model

The database has the following tables. PK means primary key (a unique ID for each row). FK means foreign key (a column that points to a row in another table).

institutions

Column	Notes
institution_id (PK)	AICTE Permanent ID
name	Official name
address, city, state	Location details
region	AICTE region
institution_type	For example Engineering, Management, Pharmacy

bureaus: bureau_id (PK), code (for example RB, PGRC), name.

users: user_id (PK), name, email, password_hash, role, bureau_id (FK, optional), region (optional), is_active.

cases

Column	Notes
case_id (PK)	Unique, for example CMP-2026-000123

Column	Notes
institution_id (FK)	Which institution the complaint is about
category	Nature of complaint
description	Complaint text
source	Student, faculty, stakeholder, other
received_on	Date of receipt
status	See section 7
current_bureau_id (FK)	Presently assigned Bureau/Cell
assigned_officer_id (FK)	Presently assigned officer
last_action_summary, last_action_on	What was last done and when
next_action_required	What must happen next
expected_disposal_on	Deadline, from the SLA rules
is_escalated	True or false
overlap_flag	True if an active overlap exists

case_actions (the permanent history; rows are only ever added, never edited or deleted): action_id (PK), case_id (FK), action_type (received, assigned, remark, status_change, transfer, escalation, document_added, disposed), from_bureau_id, to_bureau_id, performed_by (FK), performed_at (date and time), remarks.

documents: document_id (PK), case_id (FK), action_id (FK), file_name, file_path, uploaded_by, uploaded_at.

case_links: link_id (PK), case_a_id (FK), case_b_id (FK), link_type (auto_overlap, manual_related, possible_duplicate), match_score, reason (plain-text explanation), review_state (suggested, confirmed, dismissed), created_at, reviewed_by.

alerts: alert_id (PK), alert_type, case_id (FK), institution_id (FK), target_bureau_id (FK), message, severity, created_at, is_read, resolved_at.

sla_rules: category, days_allowed, warn_before_days. The values are editable by the Admin.

How tables connect: one institution has many cases; one case has many actions, documents and alerts; cases are connected to each other through case_links.

7. Case Lifecycle

Statuses

Status	Meaning
Received	Logged, not yet picked up
Under Process	An officer is working on it
Pending	Waiting for information or a reply from the institution
Transferred	Handed to another Bureau/Cell (the new unit then continues as Under Process)
Escalated	Sent to higher authority
Disposed	Closed with a final remark

Typical flow

1. A case is created with institution, category and description. The system assigns the receiving unit and sets the expected disposal date from the SLA rules.
2. The overlap engine runs immediately (section 8).
3. The officer updates status, adds remarks and uploads documents. Every change creates a case_actions row with the person, time and remarks.
4. If needed, the case is transferred or escalated. This is recorded with from-unit, to-unit and reason.
5. The case is disposed with a final remark. It stays searchable forever.

8. Overlap Detection Design

This is the main innovation, so it is designed as three layers that add up to a confidence score.

When it runs

- When a case is created
- When a case is transferred or its status changes
- Every night as a full scan

Layer 1: exact rule (strongest). Look for other open cases with the same institution_id handled by a different Bureau/Cell. If found, the score is 100.

Layer 2: fuzzy name matching. This is for messy data, such as a missing or wrong ID.

- Clean each institution name: make it lowercase, remove punctuation, expand short forms (inst → institute, tech → technology, engg → engineering, univ → university) and drop filler words like “the”.
- Compare names with RapidFuzz. A similarity of 90 or more, plus the same city or state, gives a score of about 75 to 85.

Layer 3: complaint text similarity (optional). Turn complaint descriptions into numbers with TF-IDF and compare them. Strong similarity adds up to 15 points. It also detects possible duplicates inside the same unit.

Confidence levels

Score	Action
90 and above	High: show the overlap alert and link the cases automatically
70 to 89	Medium: show as “possible overlap” for an officer to confirm
Below 70	Ignore

What officials see. A banner on both cases and on the dashboard: **“Potential Overlapping Cases – Coordination Required.”** It lists the linked cases, the units, the officers and the reason (for example “Same Institution ID, different Bureaus”).

Human in the loop. An officer can Confirm or Dismiss a suggested link. A dismissed pair is never alerted again. This avoids alert fatigue and keeps people in control.

Worked example

- Institution A → complaint received by PGRC → Under Process
- Institution A → complaint received by RB → action initiated
- Result: both cases get linked with score 100, an alert goes to PGRC and RB, and each officer can see the other’s status and open a coordination remark.

9. Alerts and Timelines (SLA)

Alert	Trigger	Who gets it
Overlap	New or updated overlap found	Officers of all units involved
Pending reminder	No action for 7 days (editable)	Assigned officer

Alert	Trigger	Who gets it
Approaching deadline	Expected disposal within 3 days (editable)	Assigned officer and their unit head
Deadline breached	Expected disposal date passed	Unit head and Senior Management
Escalation	A case is escalated	Receiving authority

A scheduler runs these checks automatically (every day in the real system, and every few minutes in the demo so judges can see it work). Alerts show in a bell icon and an Alerts page. They can be marked read or resolved.

SLA timelines are assumed. The numbers below are placeholders for the prototype and should be replaced with AICTE's prescribed timelines.

Category	Days allowed (assumed)
Fee or refund related	30
Faculty related	45
Infrastructure related	45
Approval or affiliation related	60
Other	30

10. Dashboard Design

Top row: KPI cards. Total received, Pending, Under Process, Disposed, Transferred, Escalated, Overdue, and Active overlaps.

Charts

- Status split (doughnut)
- Bureau/Cell-wise workload (bar)
- Cases received versus disposed per month (line)
- Region-wise cases (bar)

Tables

- Latest overlap alerts
- Cases nearing or past deadline
- Institutions with the most open cases

All numbers come straight from the database, so they update whenever a case changes. Filters at the top (region, Bureau/Cell, date range) change everything on the page.

11. Search, Filters and Reports

Search and filters work on: institution name, AICTE Permanent ID, case ID, Bureau/Cell, region, category, status and date range. Filters can be combined.

Reports (CSV download in the prototype)

- Institution-wise complaint history
- Region-wise report
- Bureau/Cell-wise report
- Status-wise report
- Pending-case ageing (how many days each case has been open)
- Disposal rate
- Overlapping or duplicate cases
- Monthly and quarterly summary

12. Screens and Routes

Screen	Address	Purpose
Login	/login	Sign in
Dashboard	/dashboard	KPIs, charts, alerts
Case list	/cases	Search, filter, sort
New case	/cases/new	Create a complaint
Case detail	/cases/<case_id>	Details, history, linked cases, actions
Institution profile	/institutions/<id>	Details and complaint history
Overlaps	/overlaps	All suggested and confirmed overlaps
Alerts	/alerts	All alerts
Reports	/reports	Choose and download reports
Admin	/admin	Users, bureaus, SLA rules

13. Security and Accountability

- Passwords are stored only as hashes, never as plain text.
- Every page and action checks the user's role.
- Forms use CSRF protection.
- The database is accessed through SQLAlchemy, which prevents SQL injection.
- Uploaded files are limited in type and size and renamed on save.
- Secrets (keys, passwords) live in a .env file that is never uploaded to GitHub.
- The action history is append-only: there is no screen or route to edit or delete it. This gives the audit trail the PS asks for.
- Production would run over HTTPS with proper hosting and backups.

14. Project Folder Structure

```
complaint-dashboard/
  app/
    __init__.py        # creates the Flask app
    models.py          # database tables
    auth.py            # login and role checks
    routes/
      dashboard.py
      cases.py
      institutions.py
      alerts.py
      reports.py
      admin.py
    services/
      overlap.py       # overlap detection engine
      alerts.py        # alert rules
      sla.py           # deadline calculation
      reports.py       # report building
    templates/         # HTML pages
    static/            # CSS, JS, Chart.js
  scripts/
    seed_data.py       # fills the database with fake data
  tests/
  .env
  requirements.txt
  run.py
```

15. Demo Data Plan

Because real AICTE data is not available, generate with Faker: about 30 institutions (including deliberately similar names such as “ABC Institute of Technology” and “A.B.C. Inst. of Tech.”), 6 Bureaus/Cells, 15 users, and 200 cases across all statuses and regions. Include a few planted overlaps, some overdue cases and some near-deadline cases, so every feature has something to show.

16. Demo and Test Scenarios

#	Scenario	Expected result
1	PGRC and RB both hold open cases for the same Institution ID	Overlap alert, cases linked, banner shown
2	Two cases name the same college with different spellings	Medium or high suggestion with explanation
3	Officer dismisses a suggestion	It never reappears
4	A case passes its deadline	Breach alert and it appears on the dashboard as Overdue
5	Transfer a case from PGRC to RB	History shows both units, time and reason
6	Regional officer logs in	Sees only their region
7	Download institution-wise report	CSV opens correctly

17. Build Plan (3 Days)

Adjust owners to your team size. One person can hold several roles.

Day 1: foundation

- Set up the repository, virtual environment and folders
- Create the database tables and seed data
- Login with roles, case list with search and filters
- Owners: backend and database lead, frontend lead

Day 2: core features

- Case detail page, action history, transfer and escalation
- Overlap engine (exact rule first, then fuzzy matching)

- Dashboard with KPIs and charts, alerts page
- Owners: backend lead, overlap and alerts lead, frontend lead

Day 3: finish and present

- CSV reports and bug fixes
- Run all seven demo scenarios
- Architecture slide, screenshots, demo script, idea presentation
- Owners: whole team

Cut order if time runs short (cut first to last): TF-IDF text similarity, scheduler automation (run alert checks from a button instead), extra reports, region-restricted views. **Never cut:** overlap alert, dashboard, action history, search.

18. Risks and Fallbacks

Risk	Fallback
AI-generated code breaks something	Commit to Git after every working step so you can roll back
Team members do not understand the code	Learn one topic at a time; every member must be able to explain one module
Fuzzy matching gives wrong matches	Use a high threshold and require officer confirmation
Demo fails live	Keep a recorded demo video and screenshots as backup
Not enough time	Follow the cut order in section 17

19. Future Scope

- Move to PostgreSQL and deploy on a secure government-approved cloud
- Single sign-on with AICTE staff accounts
- Live integration with the AICTE grievance and approval portals so cases flow in automatically
- Email, SMS and WhatsApp notifications
- Smarter matching using language models, handling Hindi and regional-language complaints
- Predictive analytics: which institutions are likely to receive repeat complaints

- Mobile-friendly app for field officers

20. Glossary

Term	Plain meaning
Backend	The code on the server that stores, checks and calculates
Frontend	What the user sees and clicks
Route	A web address that triggers some code
Database / table / row	Organized storage; a table is like a spreadsheet sheet; a row is one record
Primary key / foreign key	A unique ID for a row / a pointer to a row in another table
ORM (SQLAlchemy)	A translator so Python code can talk to the database
Authentication / authorization	Proving who you are / deciding what you may do
SLA	The prescribed time limit to finish a case
Ageing	How many days a case has been open
Audit trail	A permanent record of who did what and when
Fuzzy matching	Finding texts that are almost the same
Scheduler	A timer that runs code automatically
CSV	A simple spreadsheet file